

DAN-Scheduler: Deterministic Three-Stage Co-Optimization of Scheduling, Memory Layout, and Pipeline Overlap for General-Purpose NPUs

Runhao Liu
Zhejiang University
Hangzhou, China

Peng Zhang
Zhejiang University
Hangzhou, China

Abstract

Neural Processing Units (NPUs) are increasingly deployed for high-throughput and memory-constrained inference, but their hierarchical on-chip memories and heterogeneous compute and data-movement engines make execution ordering, memory placement, and pipeline overlap tightly coupled. Existing compiler flows often optimize these dimensions separately, leading to excessive on-chip residency, unnecessary off-chip traffic, and underutilized pipelines. To address these issues, we present **DAN-Scheduler**, a deterministic offline scheduling and compiler optimization framework for intra-core NPU execution that co-optimizes these decisions in three stages: (1) *Memory-Pressure-Aware Topological Scheduling (MPAS)*, reshapes execution order to shorten tensor lifetimes and suppress peak residency in high-level on-chip memories. (2) *Deterministic Linear Repackaging (DLR)*, constructs conflict-free memory layouts and applies a tier-aware cost-aware spill heuristic to reduce fragmentation-induced spill traffic under limited capacity. (3) *Critical Path Enhancement (CPE)*, improves compute–DMA overlap without perturbing the memory behavior established by the first two stages.

We evaluate DAN-Scheduler on six trace-derived operator-level Directed Acyclic Graphs (DAGs) collected from a real Davinci NPU and replayed on a generalized NPU execution model. Against four strong external baselines, DAN-Scheduler achieves the best or tied-best result on all 24 workload–metric cells, reducing peak memory, extra DDR traffic, spill count, and makespan by **18.3%, 20.4%, 14.2%, and 16.3% on average** over the best external competitor, respectively. Relative to the original schedule, it further reduces the same four metrics by **38.3%, 62.0%, 64.9%, and 57.5% on average**. These results show that deterministic stage-wise co-optimization is an effective and broadly applicable compiler optimization strategy for memory-constrained NPU execution. Our code and data are available at the following anonymous repository: <https://anonymous.4open.science/r/MICRO2026-5C74>

Keywords

Neural Processing Units, Intra-Core Scheduling, Compiler Optimization, Memory Hierarchy, Static Memory Planning, Spill-Aware Optimization, Compute–DMA Overlap, Accelerator Architecture

1 Introduction

Deep neural network inference is increasingly deployed in edge and embedded settings with tight power and memory budgets, driving

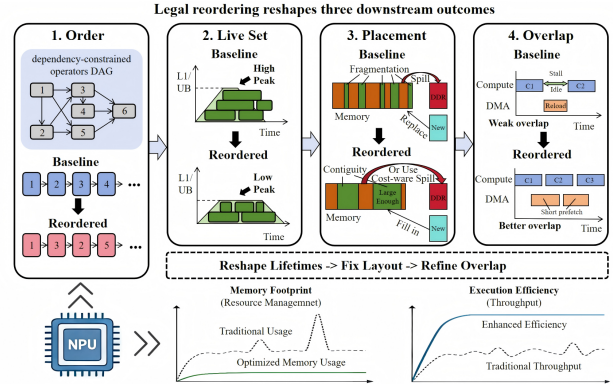


Figure 1: Coupled decisions in intra-core NPU execution.

the adoption of general-purpose Neural Processing Units (NPUs) for vision and other latency-sensitive workloads [16, 29, 30, 35]. Modern NPUs combine heterogeneous compute and data-movement engines with hierarchical on-chip memory, so performance depends on coordinating computation, communication, and storage rather than on raw parallelism alone [4, 11, 33]. As models and deployments scale, this coordination problem is increasingly limited by the memory wall [9, 19].

For intra-core execution, three decisions are tightly coupled: execution ordering, on-chip memory layout, and compute–DMA pipeline overlap. Reordering operators reshapes buffer lifetimes and the peak live set; the resulting live-set shape determines whether a conflict-free placement exists in limited on-chip memory; and placement or spill behavior, in turn, affects overlap opportunities. As illustrated in Figure 1, poor ordering can inflate residency and spill traffic, while aggressive overlap can perturb memory behavior. The key challenge is therefore to coordinate these three axes under a single compiler view.

Table 1 shows that existing approaches cover only subsets of the capabilities most relevant here: *Determinism*, *Order Shaping*, *Layout Control*, *Spill-Aware Off-Chip Reduction*, and *Pipeline Overlap*. Here, *Order Shaping* means explicitly changing execution order; *Layout Control* means explicitly managing on-chip placement or tightly coupled memory planning; *Spill-Aware Off-Chip Reduction* means explicitly reducing spills, tensor replacement, or off-chip accesses; and *Pipeline Overlap* means explicitly improving compute–DMA or broader pipeline concurrency, while *Partial* denotes limited or indirect support. HopScotch emphasizes layout-aware mapping [22, 34]; Optimus focuses on graph-level fusion [3]; Block Group

Table 1: Comparison of representative NPU optimization techniques along five qualitative dimensions.

Method	Deterministic	Order Shaping	Layout Control	Spill-Aware Off-Chip Reduction	Pipeline Overlap
HopScotch [22, 34]	✓	×	×	Partial	×
Optimus [3]	✓	Partial	×	Partial	×
Block Group Scheduling [23]	Partial	Partial	Partial	Partial	×
Flexer [13, 27]	×	✓	×	×	✓
Heterogeneous Compilation [4]	✓	×	×	×	×
HMCOS [37]	✓	✓	×	×	×
COSMA [25]	✓	✓	✓	✓	×
MAGIS [5]	✓	✓	Partial	✓	×
EDA [28]	✓	✓	Partial	✓	×
DAN-Scheduler (Ours)	✓	✓	✓	✓	✓

Scheduling partially addresses capacity-aware scheduling and allocation [23]; Flexer targets dynamic out-of-order execution in multi-NPU settings [13, 27]; and Cao’s heterogeneous CPU–NPU compilation framework focuses on cross-processor compilation rather than the intra-core co-optimization problem studied here [4]. More broadly, memory-aware scheduling, rematerialization, static memory planning, and compiler infrastructure improve important pieces of the problem, but usually optimize a single dominant objective rather than deterministic co-optimization across all three intra-core dimensions [1, 2, 6, 14, 15, 17, 18, 21, 25, 26, 31, 36].

Motivated by this gap, we present **DAN-Scheduler**, a deterministic offline scheduling and compiler optimization framework for intra-core NPU execution. DAN-Scheduler performs stage-wise co-optimization with explicit constraint propagation: *Memory-Pressure-Aware Topological Scheduling (MPAS)* reduces harmful lifetime overlap, *Deterministic Linear Repackaging (DLR)* fixes a conflict-free tiered layout and applies a tier-aware cost-aware spill heuristic, and *Critical Path Enhancement (CPE)* improves compute–DMA overlap without changing the memory behavior fixed by the first two stages. This order is deliberate: lower residency enlarges the feasible layout space, and stable layout must precede timing refinement.

We evaluate DAN-Scheduler on six trace-derived operator-level Directed Acyclic Graphs (DAGs) collected from a real Davinci NPU and replayed on a generalized NPU execution model. Against four strong external baselines, DAN-Scheduler achieves the best or tied-best result on all 24 workload–metric cells, reducing peak memory, extra Double Data Rate (DDR) traffic, spill count, and makespan by 18.3%, 20.4%, 14.2%, and 16.3% on average over the best external competitor, respectively. Relative to the original schedule, it further reduces the same four metrics by 38.3%, 62.0%, 64.9%, and 57.5% on average. These results show that deterministic stage-wise co-optimization is an effective compiler strategy for memory-constrained NPU execution.

We list our contributions as follows:

- We **identify and formalize intra-core NPU execution as a strongly coupled compiler problem**, in which execution ordering, memory layout, and compute–DMA overlap cannot be optimized independently without incurring memory or timing regressions.
- We propose **DAN-Scheduler**, a **deterministic three-stage compiler pass** that coordinates scheduling, tiered layout,

and overlap refinement through **MPAS, DLR, and CPE**, enabling lower peak residency, conflict-free memory planning, and overlap-aware timing refinement under fixed memory behavior.

- We evaluate DAN-Scheduler on **six trace-derived operator-level DAGs collected from a real Davinci NPU and replayed on a generalized NPU execution model**. Against **four strong external baselines**, DAN-Scheduler achieves the **best or tied-best result on all 24 workload–metric cells**, showing consistent gains in peak memory, extra DDR traffic, spill count, and makespan.

2 Background

2.1 Intra-Core Execution Model

Modern general-purpose NPUs integrate heterogeneous compute engines, dedicated data-movement engines, and hierarchical on-chip memories within a single core. As shown in Figure 2, a typical core combines a matrix-oriented compute engine (Cube Engine), a vector-oriented compute engine (Vector Engine), upper-level storage such as the Level-1 On-Chip Buffer and the Shared Unified Buffer, and multiple DMA engines that move data across on-chip memories, off-chip memory, and neighboring cores. Because computation and movement are handled by different engines, throughput depends on coordinating execution, communication, and storage rather than on arithmetic parallelism alone [4, 11, 33].

The memory hierarchy further shapes execution behavior. Upper-level buffers are larger but more globally shared, whereas local scratchpads are smaller and closer to compute. Figure 3 instantiates this model with Off-Chip DDR memory, Memory Transfer Engines (MTE2 and MTE3), the Level-1 (L1) Buffer, the Unified Buffer (UB), and the L0A/L0B/L0C tiers. This Davinci-derived abstraction preserves the hierarchical, engine-decoupled execution style common to general-purpose NPUs while remaining concrete enough to support compiler decisions over ordering, placement, and overlap. As a result, intra-core execution is often limited by data orchestration and memory pressure rather than raw compute throughput [4, 9, 11, 33].

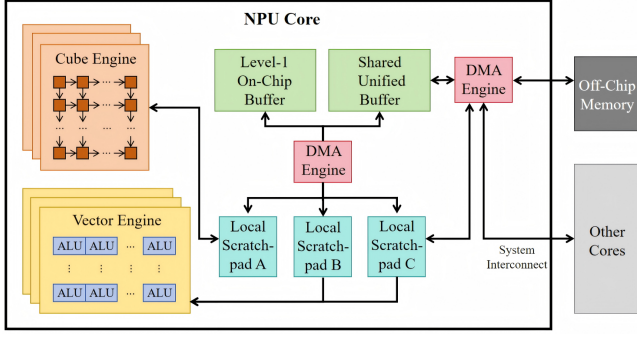


Figure 2: Generic intra-core execution model of a general-purpose NPU.

2.2 Operator-Level Execution Graph and Buffer Lifetime

DAN-Scheduler operates on an operator-level DAG in which nodes represent execution or memory-management events and edges represent precedence constraints. This makes *buffer lifetime* explicit: a buffer is live from allocation to release, and overlap among live buffers determines instantaneous on-chip residency. Peak memory pressure therefore depends more on temporal lifetime overlap than on aggregate tensor volume. A schedule that keeps many long-lived tensors live at the same time can severely stress upper-level memories even when total workload size is unchanged [1, 15, 18].

The same lifetime view explains off-chip traffic. If a required buffer cannot be placed in its designated on-chip tier, the system must spill or reload data. Conversely, if non-overlapping lifetimes can be packed into a conflict-free layout, the same hardware incurs substantially lower off-chip pressure. Prior work addresses parts of this problem through rematerialization, static memory planning, and joint scheduling-allocation optimization [2, 21, 24–26], whereas DAN-Scheduler focuses on deterministic intra-core co-optimization for memory-constrained NPUs.

2.3 Why Separate Optimization Is Insufficient

Figure 3 illustrates why scheduling, memory layout, and pipeline overlap are tightly coupled. The L1 Buffer and the UB accumulate the effects of long-lived tensor overlap, so MPAS targets execution order to reduce harmful lifetime concurrency and suppress peak residency in these upper levels. Once an order is fixed, DLR determines whether the induced lifetimes can be embedded into a conflict-free tiered layout and how spills should be handled under capacity or fragmentation pressure. CPE then refines local timing across MTE2/MTE3 and the compute engines to improve compute-DMA overlap after memory behavior has been fixed.

These interactions motivate a coupled compiler formulation rather than three separate local optimizations. The next section formalizes this dependency structure and the resulting stage-wise objective hierarchy.

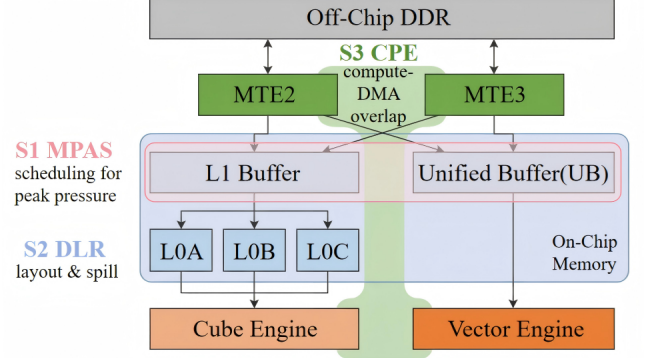


Figure 3: Davinci-derived target execution model and stage-to-resource mapping for DAN-Scheduler.

3 Problem Formulation

The interactions above suggest that intra-core NPU compilation should be formulated as a coupled optimization problem. On modern NPUs, legal execution order determines lifetime overlap, lifetime geometry constrains feasible on-chip embedding, and layout or spill behavior affects the timing space available for compute-DMA overlap [1, 2, 4, 9, 11, 15, 18, 21, 24–26, 33]. We therefore formalize DAN-Scheduler as a deterministic offline compiler pass over an operator-level DAG with a stage-wise objective hierarchy, which is realized concretely in Section 4.

3.1 Problem Scope and Graph Representation

We consider *offline*, *single-core*, and *intra-core* compilation for a target NPU execution model with heterogeneous compute engines, dedicated DMA engines, and a hierarchical on-chip memory system. The compiler input is an operator-level DAG

$$G = (V, E), \quad (1)$$

where each vertex $v \in V$ denotes an execution or explicit memory-management event, and each edge $(u, v) \in E$ denotes a precedence constraint.

Each vertex carries pipeline and cycle metadata, together with buffer-usage information. Let $\mathcal{B}$ denote the buffer set. For each $b \in \mathcal{B}$, let s_b be its size, $\tau(b) \in \mathcal{M}$ its designated home tier, and $a_b, f_b \in V$ its unique allocation and release vertices. The memory-tier set is

$$\mathcal{M} = \{L1, UB, L0A, L0B, L0C\}, \quad (2)$$

with capacity vector $C_{\text{mem}} = \{C_m\}_{m \in \mathcal{M}}$. Since the upper-level tiers dominate the target bottleneck, we distinguish

$$\mathcal{M}_{\text{pk}} = \{L1, UB\}. \quad (3)$$

A legal schedule is a bijection $\sigma : V \rightarrow \{1, \dots, |V|\}$ that preserves precedence, and the feasible schedule set is

$$\mathcal{T}(G) = \{\sigma \mid \sigma \text{ is bijective and } \sigma(u) < \sigma(v), \forall (u, v) \in E\}. \quad (4)$$

Given $\sigma \in \mathcal{T}(G)$, the lifetime interval of buffer b in schedule-position space is

$$I_b(\sigma) = [\sigma(a_b), \sigma(f_b)] \cap \mathbb{Z}. \quad (5)$$

This position-space definition is sufficient because Stage-1 cares about *which* events overlap, not yet about exact cycle-level timing. Consistent with prior memory-aware scheduling work, peak pressure is therefore driven by temporal lifetime overlap rather than total tensor volume [1, 15, 18].

For any tier $m \in \mathcal{M}$ and schedule position k , let

$$R_m(k; \sigma) = \sum_{b \in \mathcal{B}: \tau(b)=m} s_b \cdot \mathbf{1}[k \in \mathcal{I}_b(\sigma)]$$

denote the resident bytes in tier m . The Stage-1 peak objective is then

$$M_{\max}(\sigma) = \max_{1 \leq k \leq |V|} \sum_{m \in \mathcal{M}_{\text{pk}}} R_m(k; \sigma), \quad (6)$$

and we use the weighted accumulated pressure

$$\Phi(\sigma) = \sum_{k=1}^{|V|} \sum_{m \in \mathcal{M}_{\text{pk}}} \omega_m R_m(k; \sigma), \quad \omega_m > 0 \quad (7)$$

as a secondary signal that prefers schedules with a less persistent upper-tier live set.

3.2 Layout, Spill, and Timing Abstraction

Given a Stage-1 order, Stage-2 fixes a deterministic tiered layout $\mathbf{x}$ and, when direct placement fails because of capacity pressure or fragmentation, inserts explicit SPILL_OUT/SPILL_IN vertices together with reuse constraints to form an augmented DAG $G' = (V', E')$. We omit the full address-domain and disjunctive placement constraints here, since they are constructed explicitly in Section 4.3; for problem formulation, the key objects are the resulting spill plan Λ and the augmented graph.

Each spill event $e \in \Lambda$ transfers q_e bytes and incurs affine latency

$$\ell_e = \alpha_{\text{fix}} + \beta_{\text{byte}} q_e, \quad (8)$$

which captures both fixed transfer overhead and size-dependent movement cost. We summarize spill overhead by total extra DDR traffic $D_{\text{DDR}}(\Lambda)$, spill count $N_{\text{spill}}(\Lambda)$, and inserted serialization cost $C_{\text{spill}}(\Lambda) = \sum_{e \in \Lambda} \ell_e$. This separation follows prior work showing that bytes alone are insufficient to characterize memory-system overhead [2, 21, 24–26].

After Stage-2 fixes layout and spill behavior, Stage-3 optimizes timing on the augmented graph. For a refined legal order $\hat{\sigma} \in \mathcal{T}(G')$, let θ_v and $\phi_v = \theta_v + c_v$ denote the start and finish times of vertex v . The feasible timing set is

$$\Theta(\hat{\sigma}, G') = \left\{ \theta \left| \begin{array}{ll} \theta_v \geq \phi_u, & \forall (u, v) \in E', \\ \theta_v \geq \phi_{\text{prev}_{\hat{\sigma}}(v)}, & \forall v \text{ with } \text{prev}_{\hat{\sigma}}(v) \text{ defined} \end{array} \right. \right\}. \quad (9)$$

and the corresponding makespan is

$$T_{\text{exec}}(\hat{\sigma}, G') = \min_{\theta \in \Theta(\hat{\sigma}, G')} \max_{v \in V'} \phi_v. \quad (10)$$

Equation (10) makes explicit that timing optimization is performed *after* memory behavior has been fixed.

3.3 Hierarchical Optimization Objective

The Stage-2 layout-and-spill objective is

$$\Psi(\mathbf{x}, \Lambda \mid \sigma) = \lambda_D D_{\text{DDR}}(\Lambda) + \lambda_N N_{\text{spill}}(\Lambda) + \lambda_C C_{\text{spill}}(\Lambda), \quad (11)$$

which jointly penalizes extra traffic, spill frequency, and inserted transfer cost. More broadly, the coupled nature of scheduling, placement, and timing makes the compiler problem combinatorial rather than separable [6, 14, 17, 36].

DAN-Scheduler addresses this through a deterministic stage-wise decomposition. Stage-1 solves

$$\sigma^{(1)} \in \arg \min_{\sigma \in \mathcal{T}(G)} \langle M_{\max}(\sigma), \Phi(\sigma) \rangle, \quad (12)$$

so the first priority is peak upper-tier residency and the second is accumulated pressure.

Given $\sigma^{(1)}$, Stage-2 solves

$$(\sigma^{(2)}, \mathbf{x}^*, \Lambda^*, G^{(2)}) \in \arg \min_{\mathbf{x}, \Lambda, G'} \Psi(\mathbf{x}, \Lambda \mid \sigma^{(1)}), \quad (13)$$

that is, it fixes a deterministic layout and spill plan under the lifetime geometry induced by Stage-1.

Stage-3 is restricted to schedule refinements that preserve the Stage-2 lifetime structure:

$$\mathcal{R}(\sigma^{(2)}, G^{(2)}) = \{ \hat{\sigma} \in \mathcal{T}(G^{(2)}) \mid \mathcal{I}_b(\hat{\sigma}) = \mathcal{I}_b(\sigma^{(2)}), \forall b \in \mathcal{B} \}. \quad (14)$$

Hence, CPE may improve overlap, but it may not perturb the memory behavior established earlier. Under this restriction, Stage-3 solves

$$\hat{\sigma}^{(3)} \in \arg \min_{\hat{\sigma} \in \mathcal{R}(\sigma^{(2)}, G^{(2)})} T_{\text{exec}}(\hat{\sigma}, G^{(2)}). \quad (15)$$

The final compiler output is therefore the deterministic execution plan

$$\Pi^* = \mathcal{F}(G, C_{\text{mem}}) = (\sigma^{(1)}, \sigma^{(2)}, \hat{\sigma}^{(3)}, \mathbf{x}^*, \Lambda^*, G^{(2)}), \quad (16)$$

where fixed graph metadata, hardware capacities, and tie-breaking rules always produce the same plan.

4 DAN-Scheduler Design

DAN-Scheduler realizes the hierarchical objective in Section 3 through three deterministic stages with explicit interfaces. Stage-1 outputs a legal order with improved lifetime geometry, Stage-2 converts that order into a deterministic tiered layout and spill-augmented DAG, and Stage-3 performs timing refinement only within the Stage-2 feasible region.

4.1 Overview of DAN-Scheduler

Figure 4 summarizes the stage interfaces of DAN-Scheduler. The input is an operator-level DAG with compact node attributes, including operation type, execution pipeline, cycle count, buffer usage, buffer size, and memory tier. The figure also highlights the four metrics optimized hierarchically across the pipeline: peak on-chip memory, extra DDR traffic, spill count, and makespan.

The Stage-1 panel shows that MPAS operates within the legal topological set $\mathcal{T}(G)$ but reshapes harmful lifetime overlap in the upper tiers. Its output is therefore both a reordered legal schedule $\sigma^{(1)}$ and the induced lifetime structure $\{\mathcal{I}_b(\sigma^{(1)})\}_{b \in \mathcal{B}}$. The Stage-2 panel then shows how DLR converts this lifetime structure into a fixed memory plan: it first attempts deterministic best-fit placement in each buffer's home tier and invokes a tier-aware spill rule only when direct placement fails, producing the deterministic layout $\mathbf{x}^*$, spill plan Λ^* , and memory-feasible augmented DAG $G^{(2)}$. Finally, the Stage-3 panel highlights critical-chain extraction, slack-window

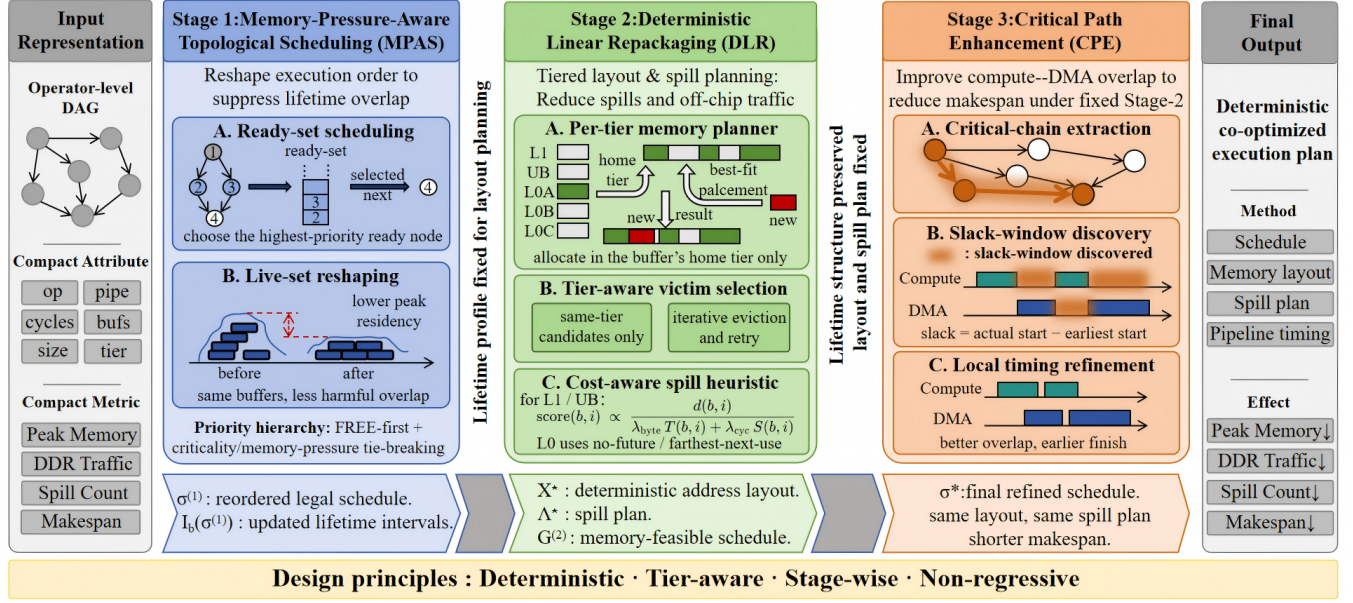


Figure 4: Overview of DAN-Scheduler. MPAS reshapes execution order, DLR fixes tiered layout and spill behavior, and CPE refines timing under the fixed Stage-2 memory plan.

discovery, and local timing refinement. CPE does not revisit layout or spill decisions; it only refines local order under the fixed Stage-2 memory behavior, yielding the final refined schedule $\hat{\sigma}^{(3)}$.

4.2 Stage-1: Memory-Pressure-Aware Topological Scheduling

Stage-1 implements Eq. (12). Its purpose is to construct a legal topological order that reduces harmful lifetime overlap in the peak-memory tiers $\mathcal{M}_{\text{pk}} = \{L1, \text{UB}\}$ [1, 15, 18]. At step k , the ready set is

$$Q_k = \{v \in V \setminus \text{dom}(\sigma_{1:k-1}) \mid (u, v) \in E \Rightarrow u \in \text{dom}(\sigma_{1:k-1})\}, \quad (17)$$

so legality is preserved by construction.

To make the FREE-first rule explicit, we define the stage-local priority level

$$\lambda(v) = \begin{cases} 0, & v = f_b \text{ for some } b \in \mathcal{B}, \\ 1, & v \text{ is neither } a_b \text{ nor } f_b, \\ 2, & v = a_b \text{ for some } b \in \mathcal{B}, \end{cases} \quad (18)$$

where smaller $\lambda(v)$ means higher priority. Within the same level, MPAS uses a one-step upper-tier pressure delta

$$\Delta_{\text{pk}}(v) = \sum_{b \in \mathcal{B}: \tau(b) \in \mathcal{M}_{\text{pk}}} s_b \cdot \eta_b(v), \quad \eta_b(v) = \begin{cases} +1, & v = a_b, \\ -1, & v = f_b, \\ 0, & \text{otherwise,} \end{cases} \quad (19)$$

and a downstream criticality score

$$h(v) = \begin{cases} c_v, & v \text{ is a sink,} \\ c_v + \max_{(v,u) \in E} h(u), & \text{otherwise.} \end{cases} \quad (20)$$

Algorithm 1 Memory-Pressure-Aware Topological Scheduling (MPAS)

Require: Original DAG $G = (V, E)$, buffer metadata

Ensure: Stage-1 schedule $\sigma^{(1)}$

- 1: Compute $h(v)$ for all $v \in V$ using Eq. (20)
- 2: Initialize indegrees and ready set Q_1
- 3: **for** $k = 1$ to $|V|$ **do**
- 4: Evaluate Eq. (21) for all $v \in Q_k$
- 5: Select v_k^* , append it to the schedule, and update indegrees
- 6: Insert any newly ready vertices into the next ready set
- 7: **end for**
- 8: **return** $\sigma^{(1)}$

The next vertex is then chosen lexicographically as

$$v_k^* \in \arg \min_{v \in Q_k} \langle \lambda(v), \Delta_{\text{pk}}(v), -h(v), \text{ord}(v) \rangle, \quad (21)$$

where $\text{ord}(v)$ is a stable input-order tie-breaker. Equation (21) directly implements the Stage-1 objective: prefer earlier frees, then lower upper-tier pressure, and only then higher downstream criticality. The overall Stage-1 procedure is summarized in Algorithm 1.

4.3 Stage-2: Deterministic Linear Repackaging with Tier-Aware Cost-Aware Spill

Given $\sigma^{(1)}$, Stage-2 solves Eq. (13). It converts the lifetime structure induced by Stage-1 into a deterministic tiered layout and introduces spill operations only when direct placement fails because of capacity pressure or fragmentation [2, 21, 24–26].

DLR scans $\sigma^{(1)}$ linearly. For each tier $m \in \mathcal{M}$, it maintains a deterministic free-segment list

$$\mathcal{F}_m^{(k)} = \left\{ [\ell_j^{(m)}, r_j^{(m)}] \right\}_{j=1}^{n_m^{(k)}}, \quad (22)$$

after processing the first $k-1$ schedule positions. When a buffer b is allocated, the feasible candidate set in its home tier is

$$C_b^{(k)} = \left\{ [\ell, r] \in \mathcal{F}_{\tau(b)}^{(k)} \mid r - \ell \geq s_b \right\}, \quad (23)$$

and if $C_b^{(k)} \neq \emptyset$, DLR chooses the unique best-fit segment

$$[\ell_b^*, r_b^*] \in \arg \min_{[\ell, r] \in C_b^{(k)}} (r - \ell - s_b), \quad x_b^* = \ell_b^*. \quad (24)$$

Eviction is event-driven: it occurs only when a required buffer is absent from its designated tier or when a new ALLOC cannot be placed by Eq. (24). For a requested buffer r at position k , DLR restricts the victim pool to the same tier,

$$\mathcal{V}_r^{(k)} = \{b \in \mathcal{B}_k^{\text{live}} \mid \tau(b) = \tau(r), b \neq r\}, \quad (25)$$

which preserves tier legality by construction.

For the near-compute tiers {L0A, L0B, L0C}, DLR uses eager flush: prefer buffers with no future use, otherwise evict the one with the farthest next use. Let

$$v(b, k) = \min \{j > k \mid b \in \mathcal{U}(v_j)\}, \quad (26)$$

where $\mathcal{U}(v_j)$ denotes the set of buffers requested or consumed by vertex v_j , and let $v(b, k) = +\infty$ if no future use exists. For the upper tiers {L1, UB}, DLR instead uses a reuse-distance-to-cost score. The reuse distance is

$$d(b, k) = \begin{cases} v(b, k) - k, & v(b, k) < +\infty, \\ +\infty, & v(b, k) = +\infty, \end{cases} \quad (27)$$

and the eviction cost is decomposed into extra traffic bytes and inserted serialization cycles:

$$T(b, k) = w_{\text{out}}(b, k) + w_{\text{in}}(b, k), \quad (28)$$

$$S(b, k) = c_{\text{out}}(b, k) + c_{\text{in}}(b, k), \quad (29)$$

where the cycle terms are derived from Eq. (8). The victim score is

$$\text{score}(b, k) = \frac{d(b, k)}{\lambda_{\text{byte}} T(b, k) + \lambda_{\text{cyc}} S(b, k) + \varepsilon}, \quad \lambda_{\text{byte}}, \lambda_{\text{cyc}} > 0, \quad (30)$$

and DLR selects

$$b^* \in \arg \max_{b \in \mathcal{V}_r^{(k)}} \text{score}(b, k). \quad (31)$$

Thus, upper-tier eviction prefers buffers whose next reuse is farther away and whose eviction incurs lower combined bandwidth and serialization overhead. Once all allocations, frees, reuse edges, and inserted spill vertices have been processed, Stage-2 returns $\sigma^{(2)}$, the fixed deterministic layout $\mathbf{x}^*$, the spill plan Λ^* , and the augmented DAG $G^{(2)}$. The overall Stage-2 procedure is summarized in Algorithm 2.

Algorithm 2 Deterministic Linear Repackaging (DLR)

Require: Stage-1 schedule $\sigma^{(1)}$, capacities C_{mem} , buffer metadata
Ensure: $\sigma^{(2)}$, deterministic layout $\mathbf{x}^*$, spill plan Λ^* , augmented DAG $G^{(2)}$

- 1: Initialize per-tier free lists $\{\mathcal{F}_m^{(1)}\}_{m \in \mathcal{M}}$
- 2: **for** each schedule position k in $\sigma^{(1)}$ **do**
- 3: **if** the current event requests or allocates a buffer b **then**
- 4: Build $C_b^{(k)}$ using Eq. (23)
- 5: **while** $C_b^{(k)} = \emptyset$ **do**
- 6: Build $\mathcal{V}_b^{(k)}$ using Eq. (25)
- 7: Evict by the L0 eager-flush rule or by Eqs. (30)–(31)
- 8: Insert the corresponding SPILL_OUT and update free lists
- 9: Rebuild $C_b^{(k)}$
- 10: **end while**
- 11: Place b by Eq. (24); insert SPILL_IN if needed
- 12: **else if** the current event frees a buffer b **then**
- 13: Return its interval to the free list of $\tau(b)$ and merge adjacent segments
- 14: **end if**
- 15: **end for**
- 16: Add reuse and spill edges to construct $G^{(2)}$
- 17: **return** $\sigma^{(2)}$, $\mathbf{x}^*$, Λ^* , $G^{(2)}$

4.4 Stage-3: Critical Path Enhancement

Given the fixed Stage-2 memory behavior, Stage-3 solves Eq. (15). Its purpose is to reduce $T_{\text{exec}}(\tilde{\sigma}, G^{(2)})$ by improving overlap between compute engines and DMA transfers while preserving the lifetime structure $\{\mathcal{I}_b(\sigma^{(2)})\}_{b \in \mathcal{B}}$, the deterministic layout $\mathbf{x}^*$, and the spill plan Λ^* .

Let $\theta^{(2)} \in \Theta(\sigma^{(2)}, G^{(2)})$ be a feasible timing assignment for the Stage-2 schedule. CPE first computes the dependency-only earliest start time

$$\underline{\theta}_v = \begin{cases} 0, & v \text{ has no predecessor in } E^{(2)}, \\ \max_{(u,v) \in E^{(2)}} \phi_u, & \text{otherwise,} \end{cases} \quad (32)$$

where $\phi_u = \theta_u + c_u$, and defines the slack

$$\text{slack}(v) = \theta_v^{(2)} - \underline{\theta}_v \geq 0. \quad (33)$$

Positive slack indicates pipeline waiting time that may be reduced by local reordering.

CPE then extracts a critical chain

$$L(C) = \sum_{v \in C} c_v, \quad C^* \in \arg \max_{C \in \mathfrak{P}(G^{(2)})} L(C), \quad (34)$$

where $\mathfrak{P}(G^{(2)})$ denotes the set of valid directed paths in $G^{(2)}$. Around each critical-chain vertex v_i , CPE then opens a local neighborhood of width W :

$$\mathcal{N}_W(v_i) = \{u \in V^{(2)} \mid |\sigma_{\text{cur}}(u) - \sigma_{\text{cur}}(v_i)| \leq W, u \notin C^*\}. \quad (35)$$

A candidate schedule $\tilde{\sigma}$ is accepted only if

$$\tilde{\sigma} \in \mathcal{R}(\sigma^{(2)}, G^{(2)}) \quad \text{and} \quad T_{\text{exec}}(\tilde{\sigma}, G^{(2)}) < T_{\text{exec}}(\sigma_{\text{cur}}, G^{(2)}). \quad (36)$$

The first condition enforces non-regressive feasibility; the second enforces strict timing improvement. Together they ensure that CPE

Algorithm 3 Critical Path Enhancement (CPE)

Require: Stage-2 schedule $\sigma^{(2)}$, augmented DAG $G^{(2)}$, fixed layout $\mathbf{x}^*$, fixed spill plan Λ^* , neighborhood width W

Ensure: Refined schedule $\hat{\sigma}^{(3)}$

```

1: Initialize  $\sigma_{\text{cur}} \leftarrow \sigma^{(2)}$  and compute a feasible timing assignment
2: for each local refinement pass do
3:   Compute  $\theta_v$  and  $\text{slack}(v)$  using Eqs. (32)–(33)
4:   Extract  $C^*$  using Eq. (34)
5:   for each  $v_i \in C^*$  do
6:     Build  $N_W(v_i)$  using Eq. (35)
7:     Enumerate local moves/swaps; accept only schedules
       satisfying Eq. (36)
8:   end for
9: end for
10: return  $\hat{\sigma}^{(3)} \leftarrow \sigma_{\text{cur}}$ 

```

cannot buy speedup by changing residency, placement legality, or spill behavior. The overall Stage-3 refinement procedure is summarized in Algorithm 3.

4.5 Determinism and Complexity

The final execution plan $\Pi^* = \mathcal{F}(G, C_{\text{mem}})$ in Eq. (16) is deterministic because MPAS uses a fixed lexicographic priority with stable tie-breaking, DLR uses deterministic best-fit placement and same-tier victim selection, spill and reuse edges are inserted in the Stage-2 scan order, and CPE enumerates neighborhoods and accepts only strictly improving moves. The stage-wise decomposition also keeps overhead tractable: Stage-1 runs in $O((|V| + |E|) \log |V|)$ with a priority queue; Stage-2 performs a single linear scan with logarithmic free-list updates, giving average-case behavior $O((|V| + |\Lambda^*|) \log |B|)$; and Stage-3 is bounded by $O(PKW(|V^{(2)}| + |E^{(2)}|))$ under full timing recomputation, where P is the number of local passes, K the critical-chain length, and W the neighborhood width. We evaluate the practical compiler overhead in Section 6.3.

5 Experimental Methodology

This section summarizes the evaluation setup, unified replay protocol, and workload characteristics. Following Section 3, we evaluate all methods on operator-level DAGs $G = (V, E)$ and report peak upper-level residency $M_{\text{max}}(\sigma)$ and end-to-end makespan T_{exec} under a common replay-based methodology. The goal here is to establish trace realism, methodological consistency, and workload diversity before presenting comparative results.

5.1 Trace Collection and Replay Setup

We evaluate DAN-Scheduler on six trace-derived operator-level DAGs collected from a real Davinci NPU execution flow and replayed on a generalized NPU execution model. The workloads cover three representative operator families common in modern inference pipelines—Convolution, FlashAttention, and Matmul—with two trace instances per family, denoted as Case0 and Case1. This setup preserves the structure of deployment-derived traces while enabling controlled compiler evaluation under a unified execution abstraction, which is appropriate for architecture-aware compiler

studies that seek to compare scheduling and memory decisions rather than device-specific runtime noise [4, 8, 11, 14, 20, 33].

Across the six workloads, the corpus contains 82,470 nodes, 165,562 directed edges, 25,920 unique buffers, and a total allocation volume of 2,377,703. Case0 and Case1 denote two trace-derived slices within the same workload family; in the current set, Case1 generally exhibits larger graph scale and stronger memory pressure. All methods use the same operator-level intermediate representation, whose node attributes include Id, Op, BufId, Size, Type, Cycles, Pipe, and BuFs, while edges encode DAG precedence constraints. The replayed target hierarchy contains the five on-chip tiers $\{L1, UB, L0A, L0B, L0C\}$ defined in Section 3. Public baselines use official implementations or adapters when available; otherwise, we use paper-faithful reimplementations. All outputs are then re-evaluated under the same replay backend for fairness.

Data integrity is important for the later memory analysis. Each buffer is associated with a symmetric ALLOC/FREE pair, so the lifetime interval $I_b(\sigma)$ is well-defined under any legal schedule $\sigma \in \mathcal{T}(G)$. This guarantees a feasible topological traversal, allows Stage-2 to interpret the Stage-1 order as a valid lifetime geometry for deterministic placement and spill planning, and makes the methodology explicitly lifetime-faithful rather than merely graph-based. This distinction matters because peak residency is governed primarily by temporal overlap rather than by total tensor volume alone [1, 15, 18].

In the replay backend, peak upper-level residency is evaluated by a linear scan over the scheduled sequence. Whenever the current event allocates a buffer whose home tier belongs to $\mathcal{M}_{\text{pk}} = \{L1, UB\}$, its size is added to the current residency; when the matching FREE event is reached, the same size is removed. The maximum value observed during this scan is reported as $M_{\text{max}}(\sigma)$, matching Eq. (6). The same replay backend also preserves execution-pipeline and cycle metadata so that Stage-2 can construct the deterministic layout $\mathbf{x}^*$ and spill plan Λ^* , and Stage-3 can evaluate $T_{\text{exec}}(\hat{\sigma}, G^{(2)})$ under fixed Stage-2 memory behavior.

Finally, all baselines and DAN variants are compared under the same replay conditions. We do not mix method-specific simulators, private runtime counters, or incompatible memory models. Instead, every method is projected to the same operator-level graph abstraction and re-evaluated under the same target tier model and metric definitions. This unified replay protocol makes the comparison reflect compiler decisions over ordering, layout, spill behavior, and timing rather than differences in measurement infrastructure [2, 20, 21, 25, 26].

5.2 Workloads and Structural Characterization

Although all six workloads come from the same Davinci-derived execution setting, they differ substantially in graph structure, memory-event density, and operator composition. Figure 5 summarizes these properties. Figure 5a shows that the workload set spans markedly different node and edge counts rather than a narrow cluster of similarly sized instances; the Case1 variants are generally larger and structurally heavier than their corresponding Case0 variants. Figure 5b reports ALLOC/FREE event counts and shows that lifecycle events remain tightly paired within each workload, which confirms explicit 1:1 lifecycle closure and supports precise lifetime

analysis and deterministic reuse reasoning. Figure 5c shows that the workloads also differ in operator mix, so the evaluation varies not only in scale but also in execution character.

Figure 6 complements this structural view with two memory-oriented characterizations. Figure 6a plots the empirical cumulative distribution function (ECDF) of ALLOC sizes and shows that allocation granularity differs substantially across workload families and cases, which directly affects fragmentation pressure in Stage-2. Figure 6b shows the distribution of buffer lifetimes. Broader or longer-tailed lifetime distributions create more persistent live-set pressure, which increases peak upper-level residency and makes conflict-free layout harder; shorter and more concentrated lifetimes are generally easier to pack. These differences explain why the same replay-based framework exposes distinct peak-memory, fragmentation, and spill behaviors across workloads and also motivate the stage-wise design of DAN-Scheduler: first reshape lifetime overlap, then stabilize the induced layout, and only then refine timing.

Together, Figures 5 and 6 show that the workloads are trace-derived, measured under a unified replay backend, and diverse enough to stress different compiler decisions. This makes the later comparison meaningful because observed differences can be attributed to the compiler rather than to inconsistent infrastructure.

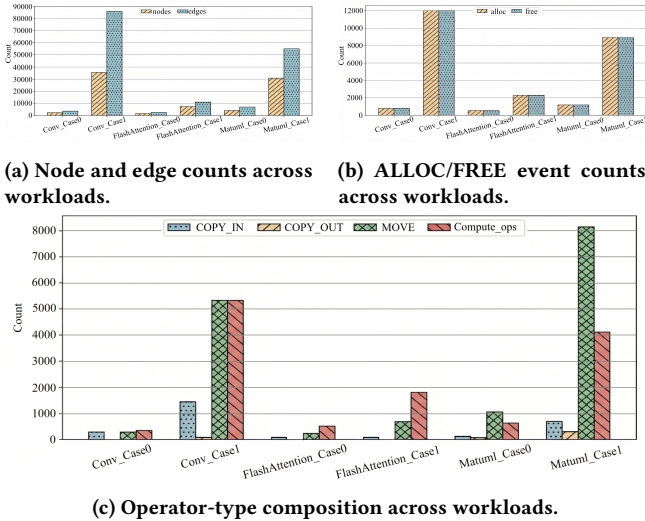
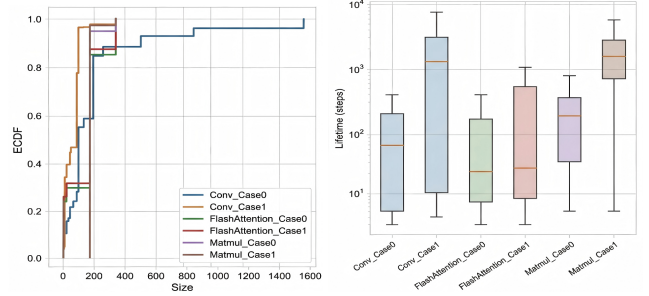


Figure 5: Structural overview of the workloads, showing diversity in graph structure and memory-management.

6 Evaluation

Before presenting the detailed results, we briefly summarize the evaluation goal and structure. Our evaluation is organized around three questions. First, does DAN-Scheduler outperform strong external baselines in overall memory, traffic, spill, and runtime behavior under a unified replay-based setting? Second, can the gains be cleanly attributed to the three stages—MPAS, DLR, and CPE—rather than to a single dominant optimization? Third, is the full pipeline



(a) ECDF of ALLOC sizes across (b) Buffer-lifetime distributions across workloads.

Figure 6: Memory-oriented characteristics of the six trace-derived workloads.

practical as an offline compiler pass? Accordingly, Section 6.1 compares DAN against external baselines, Section 6.2 provides stage-wise attribution and ablation evidence, and Section 6.3 examines compiler overhead and practical deployment implications.

6.1 Overall Comparison Against Baselines

We first compare DAN-Scheduler against four strong external baselines: HMCOS [37], COSMA [25], MAGIS [5], and EDA [28]. To make this comparison fair under a unified operator-level setting, all methods are projected to the same operator-level intermediate representation and re-evaluated under the same target tier model and generalized NPU replay backend, rather than under method-specific simulators or private runtime counters. In particular, HMCOS is used as a public-code-structure-derived scheduling baseline, COSMA is instantiated as a paper-faithful solver-free reimplement structure, MAGIS is used as a MAGIS-adapted reordering baseline under the same replay backend, and EDA is instantiated as an EDA-style traversal baseline derived from the public method description. This evaluation protocol ensures that the comparison reflects differences in compiler decisions over ordering, memory behavior, and timing, rather than differences in measurement infrastructure.

Table 2 shows that DAN achieves the best or tied-best result on all 24 workload-metric cells. Relative to the strongest external baseline on each workload and metric, DAN reduces peak memory, extra DDR traffic, spill count, and makespan by 18.3%, 20.4%, 14.2%, and 16.3% on average, respectively. The largest gains appear on the convolution workloads: on Conv_Case0, DAN reduces normalized peak memory from 0.734 to 0.340 and extra DDR traffic from 0.535 to 0.307; on Conv_Case1, it lowers normalized spill count from 0.514 to 0.333, indicating that Stage-1 live-set reshaping and Stage-2 tier-aware placement work particularly well under heavier memory pressure.

The FlashAttention workloads show that DAN improves memory-oriented and timing-oriented metrics together: although EDA is typically the strongest external competitor on traffic and makespan, DAN still improves all four metrics. The Matmul workloads are the most competitive for peak memory: DAN ties MAGIS on $M_{\max}$ for both workloads, but still outperforms all external baselines on extra

Table 2: Main comparison against baselines. All values are normalized to *Base* / *Original* on the same workload; smaller is better. The row “ Δ vs. best ext.” reports the relative improvement of DAN over the strongest external baseline for each metric.

Workload	Method	Peak Mem/Base	Extra DDR/Base	Spill Count/Base	Makespan/Base
Conv_Case0	HMCOS	1.000	0.535	0.548	0.622
	COSMA	1.000	0.698	0.687	0.739
	MAGIS	0.734	0.736	0.389	0.730
	EDA	1.000	0.616	0.543	0.650
	DAN (ours)	0.340	0.307	0.356	0.434
	Δ vs. best ext.	53.7%↓	42.6%↓	8.5%↓	30.2%↓
Conv_Case1	HMCOS	1.000	0.592	0.616	0.652
	COSMA	1.000	0.727	0.689	0.741
	MAGIS	0.949	0.564	0.514	0.602
	EDA	1.000	0.528	0.518	0.575
	DAN (ours)	0.758	0.407	0.333	0.441
	Δ vs. best ext.	20.1%↓	22.9%↓	35.2%↓	23.3%↓
FlashAttention_Case0	HMCOS	1.000	0.537	0.487	0.555
	COSMA	1.000	0.685	0.641	0.697
	MAGIS	0.898	0.491	0.399	0.502
	EDA	1.000	0.475	0.379	0.481
	DAN (ours)	0.720	0.395	0.352	0.426
	Δ vs. best ext.	19.8%↓	16.8%↓	7.1%↓	11.1%↓
FlashAttention_Case1	HMCOS	1.000	0.542	0.499	0.560
	COSMA	1.000	0.722	0.680	0.730
	MAGIS	0.942	0.493	0.404	0.502
	EDA	1.000	0.473	0.383	0.480
	DAN (ours)	0.788	0.398	0.361	0.429
	Δ vs. best ext.	16.3%↓	15.9%↓	5.7%↓	10.6%↓
Matmul_Case0	HMCOS	1.000	0.559	0.562	0.589
	COSMA	1.000	0.760	0.701	0.760
	MAGIS	0.563	0.602	0.510	0.599
	EDA	1.000	0.437	0.401	0.459
	DAN (ours)	0.563	0.362	0.319	0.387
	Δ vs. best ext.	0.0% (tie)	17.2%↓	20.4%↓	15.7%↓
Matmul_Case1	HMCOS	1.000	0.565	0.576	0.597
	COSMA	1.000	0.769	0.708	0.767
	MAGIS	0.531	0.634	0.541	0.627
	EDA	1.000	0.442	0.417	0.466
	DAN (ours)	0.531	0.411	0.384	0.436
	Δ vs. best ext.	0.0% (tie)	7.0%↓	7.9%↓	6.4%↓

DDR traffic, spill count, and makespan. Overall, Table 2 shows that DAN is not only better than the original schedule, but also stronger than diverse external baselines across workload families.

6.2 Stage-Wise Analysis and Attribution

6.2.1 Stage-1: Memory-Pressure-Aware Topological Scheduling (MPAS).

Table 3 isolates the contribution of MPAS to the Stage-1 objective in Eq. (6). Across the six workloads, MPAS reduces peak upper-tier residency by 83.05% on average, with the largest absolute reductions on Conv_Case1. This is consistent with the role of MPAS:

by advancing release opportunities and delaying harmful allocations whenever legality allows, it reduces overlap among long-lived buffers in the peak-memory tiers.

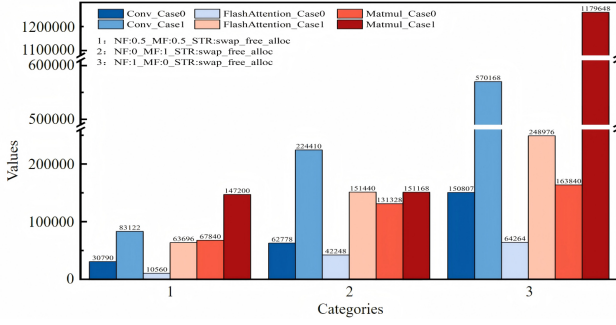
Figure 7 shows that the balanced priority weighting is more robust than either a purely memory-first or purely criticality-first rule. The former can miss opportunities to close use-release chains earlier, whereas the latter allows large live sets to accumulate. MPAS therefore improves Stage-1 not by greedily suppressing allocations, but by reshaping the legal topological order in a dependence-aware way.

6.2.2 Stage-2: Deterministic Linear Repackaging (DLR) and Spill Behavior.

Figure 8 shows that spill behavior cannot be summarized by

Table 3: Peak upper-tier on-chip memory ($M_{\max}$ over L1 and UB) before and after Stage-1 (MPAS).

Workload	Base $M_{\max}$	MPAS $M_{\max}$	Reduction
Conv_Case0	62,778	13,584	78.37%
Conv_Case1	476,790	41,884	91.22%
FlashAttention_Case0	42,248	7,820	81.49%
FlashAttention_Case1	171,664	23,196	86.50%
Matmul_Case0	131,328	27,904	78.75%
Matmul_Case1	1,048,832	189,056	81.97%
Average	—	—	83.05%

**Figure 7: Sensitivity of the Stage-1 priority weighting. Lower values indicate lower peak memory.**

spill count alone: some workloads incur many spill events without the largest traffic volume, whereas others generate fewer events but larger extra DDR movement. The heatmaps in Figure 9 explain how DLR improves this behavior: across Conv_Case0, FlashAttention_Case0, and Matmul_Case0, the address offsets progress more regularly and contiguously, indicating that DLR preserves a more stable tiered address space and reduces fragmentation-induced spills.

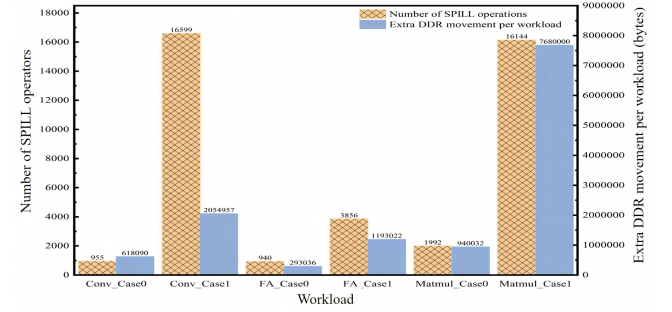
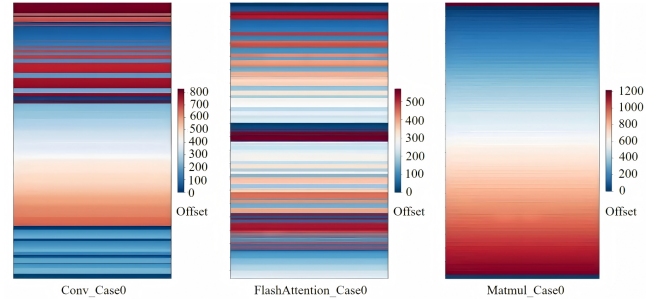
Table 4 isolates the Stage-2 spill rule. Relative to the LRU-based variant, the tier-aware cost-aware heuristic consistently reduces extra DDR traffic and spill count, with the largest improvements on Conv_Case0 for traffic and Matmul_Case0 for spill frequency. The makespan effect is mixed, which is expected: Stage-2 is designed to improve layout quality and spill quality under fixed lifetime structure, not to directly optimize T_{exec} . Its primary benefit is therefore a better deterministic layout and spill plan Λ^* , while the final timing gains are left to Stage-3.

6.2.3 Stage-3: Critical Path Enhancement (CPE). Table 5 isolates the effect of CPE after the deterministic layout x^* , spill plan Λ^* , and augmented graph $G^{(2)}$ have been fixed by Stage-2. Under this constraint, CPE consistently reduces makespan on all six workloads by 1.7%–4.2%, so the gain can be attributed directly to timing refinement and improved compute–DMA coordination rather than to hidden changes in memory behavior.

Figure 10 provides qualitative evidence for this mechanism. Across the three representative Case1 workloads, the post-CPE schedules exhibit smaller idle gaps and tighter interleaving between compute

Table 4: Effect of replacing the LRU spill rule with the tier-aware cost-aware heuristic in Stage-2. Negative values indicate reductions relative to the LRU-based variant.

Workload	DDR Change	Spill-Count Change	Makespan Change
Conv_Case0	-10.8%	-3.4%	-4.3%
Conv_Case1	-3.3%	-3.1%	+0.6%
FlashAttention_Case0	-0.3%	-0.2%	+3.8%
FlashAttention_Case1	-0.1%	-0.1%	+3.2%
Matmul_Case0	-7.2%	-10.4%	-2.8%
Matmul_Case1	-2.7%	-3.7%	+0.4%

**Figure 8: Spill count and extra DDR movement across workloads.****Figure 9: Representative offset heatmaps for three Case0 workloads illustrating DLR-generated allocation patterns.**

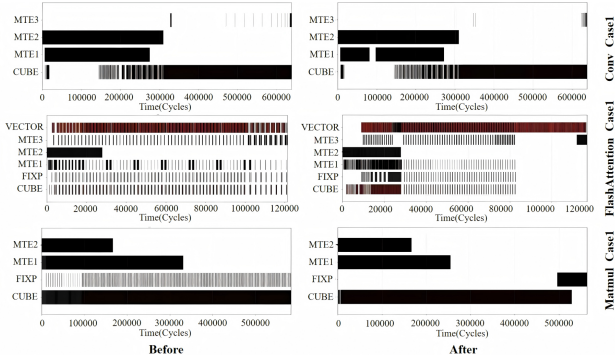
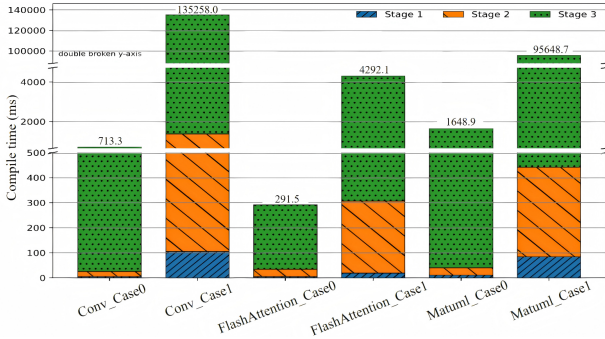
and data-movement engines. The largest reduction appears on Matmul_Case0 rather than on the largest graph overall, indicating that the usefulness of Stage-3 depends on overlap headroom and local waiting structure, not only on graph size.

6.3 Compiler Overhead and Practicality

Figure 11 evaluates the practicality of DAN as an offline compiler pass. Stage-1 remains lightweight across all workloads, Stage-2 adds a moderate but bounded cost to construct the deterministic layout and spill plan, and Stage-3 dominates total compile time on the largest workloads because it performs local timing refinement around critical regions.

Table 5: Isolated impact of CPE on end-to-end makespan, comparing DAN w/o CPE and DAN(full). Smaller is better.

Workload	DAN w/o CPE	DAN(full)	Makespan↓
Conv_Case0	1,559,801	1,533,691	1.7%
Conv_Case1	9,273,845	9,004,141	2.9%
FA_Case0	664,194	638,703	3.8%
FA_Case1	2,731,599	2,643,529	3.2%
Matmul_Case0	1,712,198	1,639,805	4.2%
Matmul_Case1	14,261,042	13,841,233	2.9%


Figure 10: Representative Gantt charts before and after CPE for three Case1 workloads. CPE reduces idle gaps and improves overlap under the fixed Stage-2 memory plan.

Figure 11: Stage-wise compile-time breakdown of DAN(full).

This profile matches the intended deployment model. DAN is an offline compiler optimization framework rather than an online runtime scheduler, so the relevant question is whether the extra optimization cost is concentrated in the stage that directly improves end-to-end timing. Figure 11 shows that this is the case: the most expensive stage is also the optional refinement stage. DAN without CPE can therefore serve as a faster compile mode when turnaround matters, while DAN(full) remains the best-performing mode when maximum execution efficiency is preferred.

7 Related Work

Memory-aware and inter-operator scheduling. Prior work shows that execution order strongly affects memory pressure and utilization, but usually optimizes only part of the problem. Representative examples include memory-aware scheduling, HMCOS, AINNS, and recent inter-operator scheduling work [1, 10, 37, 44], as well as HopScotch, Stream, Block Group Scheduling, EDA, and Flexer [13, 22, 23, 27, 28, 34]. These works confirm the importance of order shaping, but they do not first fix a deterministic tiered layout and spill plan and then refine timing on top of that fixed memory behavior.

Memory planning, spill reduction, and rematerialization. A second line of work addresses memory feasibility through joint scheduling-allocation, static planning, or recomputation. COSMA and MAGIS optimize memory behavior through joint or coordinated graph decisions [5, 25]. Telamalloc, generic memory-planning methods, MODEL, and recent static planning frameworks study fragmentation- or capacity-aware placement [12, 21, 24, 26, 32]. Efficient rematerialization systems such as TASO, Efficient Rematerialization, Checkmate, MOCCASIN, and T-Control reduce residency by trading computation or movement for memory [2, 15, 18, 38, 39]. In contrast, DAN-Scheduler couples a deterministic layout and tier-aware spill plan with a lifetime-shaped execution order, and preserves that Stage-2 memory behavior during later timing refinement.

NPU compiler, mapping, and simulation frameworks. A third line of work provides compiler and evaluation infrastructure around accelerator execution. TVM, learning-based tensor-program optimization, TASO, and PET advance graph transformation and tensor-program optimization [6, 7, 17, 36]; layer-wise exploration and automatic accelerator-generation frameworks study broader compiler design spaces [14, 41]; and systems such as heterogeneous CPU-NPU compilation, PIMCOMP, gem5-NVDLA, PyTorchSim, hardware-assisted NPU virtualization, and ReGate strengthen the compiler, simulator, and system stack [4, 20, 33, 40, 42, 43]. These frameworks are complementary to DAN-Scheduler, but they do not directly solve deterministic intra-core co-optimization over ordering, conflict-free layout, spill-aware off-chip reduction, and compute-DMA overlap.

8 Conclusion

DAN-Scheduler shows that memory-constrained intra-core NPU execution should be treated as an architecture-aware compiler problem rather than as separate optimizations of execution ordering, memory placement, and pipeline overlap. On general-purpose NPUs, these decisions are tightly coupled: execution order reshapes buffer lifetime overlap, lifetime geometry constrains feasible on-chip layout, and layout and spill behavior determine the overlap that can be exposed between compute and DMA. This makes deterministic stage-wise coordination the right compiler abstraction for such architectures.

Accordingly, DAN-Scheduler combines MPAS, DLR, and CPE into a deterministic offline compiler pass that first reduces harmful upper-tier residency, then fixes a conflict-free tiered layout and spill plan, and finally refines timing under fixed memory behavior. Across six trace-derived operator-level DAGs collected from a real

Davinci NPU and replayed on a generalized execution model, it consistently improves peak memory, extra DDR traffic, spill count, and makespan over both the original schedule and strong external baselines.

More broadly, DAN-Scheduler suggests a reusable compiler principle for NPUs with hierarchical memories and decoupled compute and data-movement engines: stabilize memory behavior before optimizing timing. We believe this perspective is important because it connects compiler structure directly to architectural constraints and provides a deterministic, analyzable path toward larger-scope NPU compilation. Future work will extend this direction to larger graphs, multi-core execution, and tighter runtime and system-level coordination.

References

- [1] Byung Hoon Ahn, Jinwon Lee, Jamie Menjay Lin, Hsin-Pai Cheng, Jilei Hou, and Hadi Esmaeilzadeh. 2020. Ordering chaos: Memory-aware scheduling of irregularly wired neural networks for edge devices. *Proceedings of Machine Learning and Systems* 2 (2020), 44–57.
- [2] Burak Bartan, Haoming Li, Harris Teague, Christopher Lott, and Bistra Dilkina. 2023. MOCCASIN: efficient tensor rematerialization for neural networks. In *International Conference on Machine Learning*. PMLR, 1826–1837.
- [3] Xuyi Cai, Ying Wang, and Lei Zhang. 2022. Optimus: An operator fusion framework for deep neural networks. *ACM Transactions on Embedded Computing Systems* 22, 1 (2022), 1–26.
- [4] Shan Cao, Meiling Yang, Yintao Liu, Yu Li, Beining Zhao, Xinyu Chen, and Zhiyuan Jiang. 2025. A heterogeneous cnn compilation framework for risc-v cpu and npu integration based on onnx-mlir. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [5] Renze Chen, Zijian Ding, Size Zheng, Chengrui Zhang, Jingwen Leng, Xuanzhe Liu, and Yun Liang. 2024. Magis: Memory optimization via coordinated graph transformation and scheduling for dnn. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 607–621.
- [6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594.
- [7] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. *Advances in Neural Information Processing Systems* 31 (2018).
- [8] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [9] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. 2024. Ai and memory wall. *IEEE Micro* 44, 3 (2024), 33–39.
- [10] Shuxi Guo, Zikang Xu, Jiahao Liu, Jinyi Zhang, Qi Qi, Haifeng Sun, Jun Huang, Jianxin Liao, and Jingyu Wang. 2025. Efficient Inter-operator scheduling for concurrent recommendation model inference on GPU. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. 2856–2864.
- [11] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Nupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.
- [12] Zixiao Huang, Junhao Hu, Hao Lin, Chunyang Zhu, Yueran Tang, Quanlu Zhang, Zhen Guo, Zhenhua Li, Shengen Yan, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2025. STAlloc: Enhancing Memory Efficiency in Large-Scale Model Training with Spatio-Temporal Planning. *arXiv preprint arXiv:2507.16274* (2025).
- [13] Soojin Hwang, Sunho Lee, Jungwoo Kim, Hongbeen Kim, and Jaehyuk Huh. 2023. mNPUsum: Evaluating the effect of sharing resources in multi-core NPUs. In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 167–179.
- [14] Fabrizio Indirli, Andrea Carlo Ornstein, Giuseppe Desoli, Alessandro Buschini, Cristina Silvano, and Vittorio Zaccaria. 2024. Layer-wise Exploration of a Neural Processing Unit Compiler’s Optimization Space. In *Proceedings of the 2024 10th International Conference on Computer Technology Applications*. 20–26.
- [15] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems* 2 (2020), 497–511.
- [16] Rakshith Jayanth, Neelesh Gupta, and Viktor Prasanna. 2024. Benchmarking edge AI platforms for high-performance ML inference. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.
- [17] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 47–62.
- [18] Ravi Kumar, Manish Purohit, Zoya Svitkina, Erik Vee, and Joshua Wang. 2019. Efficient rematerialization for deep networks. *Advances in Neural Information Processing Systems* 32 (2019).
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [20] Chengtao Lai and Wei Zhang. 2024. gen5-NVDLA: A simulation framework for compiling, scheduling, and architecture evaluation on AI system-on-chips. *ACM Transactions on Design Automation of Electronic Systems* 29, 5 (2024), 1–20.
- [21] Christos Lamprikos, Panagiotis Xanthopoulos, Manolis Katsaragakis, Sotirios Xydis, Dimitrios Soudris, and Franky Cathoor. 2025. Futureproof Static Memory Planning. *arXiv preprint arXiv:2504.04874* (2025).
- [22] Suhong Lee, Boyeal Kim, Yongseok Choi, and Hyuk-Jae Lee. 2025. HopScotch: A Holistic Approach to Data Layout-Aware Mapping on NPUs for High-Performance DNN Inference. *ACM Transactions on Architecture and Code Optimization* 22, 3 (2025), 1–26.
- [23] Seokho Lee, Younghyun Lee, Hyejun Kim, Taehoon Kim, and Yongjun Park. 2023. Block Group Scheduling: A General Precision-scalable NPU Scheduling Technique with Capacity-aware Memory Allocation. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [24] Maksim Levental. 2022. Memory planning for deep neural networks. *arXiv preprint arXiv:2203.00448* (2022).
- [25] Yi Li, Aarti Gupta, and Sharad Malik. 2023. Combined scheduling, memory allocation and tensor replacement for minimizing off-chip data accesses of dnn accelerators. *arXiv preprint arXiv:2311.18246* (2023).
- [26] Martin Maas, Ulysse Beaunon, Arun Chauhan, and Berkin Ilbeyli. 2022. Talamallo: Efficient on-chip memory allocation for production machine learning accelerators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 123–137.
- [27] Hyemi Min, Jungyoon Kwon, and Bernhard Egger. 2023. Flexer: Out-of-order scheduling for multi-npus. In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization*. 212–223.
- [28] Bo Ren Pao, I-Chia Chen, En-Hao Chang, and Tsung Tai Yeh. 2025. EDA: Energy-Efficient Inter-Layer Model Compilation for Edge DNN Inference Acceleration. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 563–576.
- [29] Biagio Peccerillo, Mirco Mannino, Andrea Mondelli, and Sandro Bartolini. 2022. A survey on hardware accelerators: Taxonomy, trends, challenges, and perspectives. *Journal of Systems Architecture* 129 (2022), 102561.
- [30] Arnab Raha, Deepak A Mathaikutty, Shamik Kundu, and Soumendu K Ghosh. 2025. FlexNPU: A dataflow-aware flexible deep learning accelerator for energy-efficient edge devices. *Frontiers in High Performance Computing* 3 (2025), 1570210.
- [31] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirsame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: A new ir for machine learning frameworks. In *Proceedings of the 2nd ACM SIGPLAN international workshop on machine learning and programming languages*. 58–68.
- [32] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. 2023. Model: memory optimizations for deep learning. In *International Conference on Machine Learning*. PMLR, 32618–32632.
- [33] Xiaotian Sun, Xinyu Wang, Wanqian Li, Yinhe Han, and Xiaoming Chen. 2024. PIMCOMP: an end-to-end DNN compiler for processing-in-memory accelerators. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 5 (2024), 1745–1759.
- [34] Arne Symons, Linyan Mei, Steven Coleman, Pouya Houshmand, Sebastian Karl, and Marian Verhelst. 2024. Stream: Design space exploration of layer-fused DNNs on heterogeneous dataflow accelerators. *IEEE Trans. Comput.* 74, 1 (2024), 237–249.
- [35] Tianxiang Tan and Guohong Cao. 2020. FastVA: Deep learning video analytics through edge processing and NPU in mobile. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 1947–1956.
- [36] Haojie Wang, Jidong Zhai, Mingyu Gao, Zixuan Ma, Shizhi Tang, Liyan Zheng, Yuanzhi Li, Kaiyuan Rong, Yuanyong Chen, and Zhihao Jia. 2021. {PET}: Optimizing tensor programs with partially equivalent transformations and automated corrections. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. 37–54.
- [37] Zihan Wang, Chengcheng Wan, Yuting Chen, Ziyi Lin, He Jiang, and Lei Qiao. 2022. Hierarchical memory-constrained operator scheduling of neural architecture search networks. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 493–498.
- [38] Zehua Wang, Junmin Xiao, Xiaochuan Deng, Huibing Wang, Hui Ma, Mingyi Li, Yunfei Pang, and Guangming Tan. 2026. T-Control: An Efficient Dynamic Tensor Rematerialization System for DNN Training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1949–1965.
- [39] Yuan Wen, Andrew Anderson, Valentin Radu, Michael FP O’Boyle, and David Gregg. 2020. Taso: Time and space optimization for memory-constrained DNN inference. In *2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 199–208.
- [40] Yuqi Xue and Jian Huang. 2025. ReGate: Enabling Power Gating in Neural Processing Units. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1160–1177.
- [41] Yongqi Xue, Jinlun Ji, Xinming Yu, Shize Zhou, Siyue Li, Xinyi Li, Tong Cheng, Shiping Li, Kai Chen, Zhonghai Lu, Li Li, and Yuxiang Fu. 2024. Automatic Generation and Optimization Framework of NoC-Based Neural Network Accelerator Through Reinforcement Learning. *IEEE Trans. Comput.* 73, 12 (2024), 2882–2896.
- [42] Yuqi Xue, Yiqi Liu, Lifeng Nai, and Jian Huang. 2024. Hardware-assisted virtualization of neural processing units for cloud platforms. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–16.

- [43] Wonhyuk Yang, Yunseon Shin, Okkyun Woo, Geonwoo Park, Hyungkyu Ham, Jeehoon Kang, Jongse Park, and Gwangsun Kim. 2025. PyTorchSim: A Comprehensive, Fast, and Accurate NPU Simulation Framework. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1363–1380.
- [44] Jiaqi Zhang, Xiangru Chen, and Sandip Ray. 2022. AINNS: All-Inclusive Neural Network Scheduling Via Accelerator Formalization. *IEEE Trans. Comput.* 72, 2 (2022), 559–571.